3.3 Frameworki Agentowe: Budowa wielozadaniowych agentów w LangChain lub CrewAI

Moduł 3: Budowa Aplikacji i Agenci AI

Krzysztof Rytlewski · 2026

Czym jest agent AI i co go odróżnia od zwykłego LLM?

Dotychczas pracowaliśmy z modelami w trybie **zapytanie → odpowiedź** – jeden prompt, jedna odpowiedź, koniec. Agent AI to system, który **autonomicznie planuje i wykonuje sekwencje działań**, aby osiągnąć złożony cel.

Kluczowe cechy agenta:

- **Planowanie** – rozkłada złożone zadanie na podzadania i decyduje o kolejności wykonania.
- **Używanie narzędzi** – może przeszukiwać internet, czytać pliki, wykonywać kod, odpytywać bazy danych, wywoływać API.
- **Pamięć** – przechowuje wyniki pośrednich kroków i uwzględnia je w dalszym planowaniu.
- **Iteracja** – ocenia wyniki swoich działań i koryguje kurs przy błędach.

Analogia i pętla ReAct

Analogia: Asystent odpowiadający na jedno pytanie vs. pracownik z celem biznesowym: *„Znajdź trzech partnerów technologicznych, sprawdź ich ofertę, porównaj ceny i przygotuj raport w Excelu"*. Pracownik sam planuje kroki i przynosi gotowy wynik.

Pętla ReAct (*Reasoning + Acting*) – podstawowy wzorzec agentowy:

```
1. Thought: Co muszę zrobić żeby osiągnąć cel?
2. Action: wywołaj narzędzie X z parametrami Y
3. Observation: wynik narzędzia X
4. Thought: Co mówi wynik? Czy cel jest osiągnięty?
5. → jeśli nie: wróć do kroku 2
6. → jeśli tak: Final Answer
```

LangChain Agents – budowa agenta z narzędziami

```
const { Calculator } = require('@langchain/community/tools/calculator');
const tools = [new Calculator(), getWeatherTool];
const llm = new ChatOpenAI({ model: 'gpt-4o', temperature: 0 });

const agent = await createReactAgent({ llm, tools });
const executor = new AgentExecutor({ agent, tools, verbose: true });

const result = await executor.invoke({
  input: 'Jaka jest pogoda w Warszawie i Krakowie? Która temperatura jest wyższa?',
});
```

Flaga `verbose: true` pokazuje pełną pętlę Thought/Action/Observation – niezbędna przy debugowaniu.

Narzędzia wbudowane w LangChain Community

Narzędzie	Klasa	Opis
Kalkulator	`Calculator`	Obliczenia matematyczne
Wyszukiwarka	`TavilySearchResults`	Wyszukiwanie w internecie
Wikipedia	`WikipediaQueryRun`	Zapytania do Wikipedii
Wykonanie kodu	`NodeCodeInterpreter`	Uruchamianie kodu JS
Zapytania HTTP	`RequestsGetTool`	Wywołanie dowolnego URL

Bogaty ekosystem narzędzi oznacza, że większość integracji jest już gotowa – własne narzędzia piszesz tylko dla specyficznej logiki biznesowej.

LangGraph – agenci stanowi i przepływy z pętlami

LangGraph rozszerza LangChain o budowanie agentów jako **grafów stanów** (*stateful graphs*). Umożliwia definiowanie złożonych przepływów z rozgałęzieniami, pętlami i synchronizacją wielu agentów.

Kiedy używać LangGraph zamiast prostego agenta:

- Gdy agent musi podejmować różne ścieżki w zależności od stanu (np. eskalacja do człowieka przy ryzykownych akcjach).
- Gdy potrzebujesz pełnej kontroli nad przepływem i możliwości przerwania (*human-in-the-loop*).
- Gdy budujesz system wieloagentowy.

LangGraph – podstawowy graf agenta

```
const workflow = new StateGraph({ channels: agentState })
  .addNode('agent', agentNode)
  .addNode('tools', new ToolNode(tools))
  .setEntryPoint('agent')
  .addConditionalEdges('agent', shouldContinue)
  .addEdge('tools', 'agent');

const app = workflow.compile();
```

Checkpointing – zapis stanu grafu w MongoDB umożliwia wznowienie przerwanego agenta:

```
const checkpointer = new MongoDBSaver({
  client: mongoClient,
  dbName: 'agent-state'
});
const app = workflow.compile({ checkpointer });
```

CrewAI – orkiestracja zespołów agentów

CrewAI opiera się na metaforze załogi: zamiast jednego agenta-generalisty, tworzysz **zespół wyspecjalizowanych agentów**, z których każdy ma konkretną rolę, cel i zestaw narzędzi.

Kluczowe pojęcia:

- **Agent** – ma rolę (`role`), cel (`goal`), kontekst (`backstory`) i przypisane narzędzia.
- **Task** – konkretne zadanie przypisane do agenta z opisem i oczekiwanym wyjściem.
- **Crew** – zespół agentów i zadań z określonym procesem wykonania.
- **Process** – sposób koordynacji: `sequential` (jeden po drugim) lub `hierarchical` (manager deleguje).

CrewAI – przykład pipeline analizy konkurencji

```
const researchTask = new Task({
  description: 'Zbierz informacje o 3 głównych konkurentach: funkcje, ceny, opinie',
  expectedOutput: 'Ustrukturyzowany raport z danymi o każdym konkurencie',
  agent: researcher,
});

const reportTask = new Task({
  description: 'Przygotuj raport SWOT i rekomendacje strategiczne',
  agent: writer,
  context: [researchTask], // wynik poprzedniego zadania jako wejście
});

const crew = new Crew({
  agents: [researcher, writer],
  tasks: [researchTask, reportTask],
  process: Process.sequential,
});
const result = await crew.kickoff({ topic: 'Narzędzia do zarządzania projektami' });
```

Zasady projektowania narzędzi agentów

Kluczem do skutecznego działania agenta jest precyzyjny opis narzędzia – model decyduje kiedy i jak je wywołać wyłącznie na podstawie nazwy i opisu.

1. **Jedna odpowiedzialność** – narzędzie robi jedną rzecz dobrze.
2. **Precyzyjny opis** – wyjaśnij kiedy NIE używać narzędzia, nie tylko kiedy używać.
3. **Walidacja wejść** – agent może podać nieprawidłowe parametry; zawsze waliduj przez Zod.
4. **Czytelne wyjście** – zwracaj tekst zrozumiały dla modelu, nie surowy zagnieżdżony JSON.

```
const searchUsersTool = new DynamicStructuredTool({
  schema: z.object({
    query: z.string().describe('Tekst do wyszukania w imieniu lub emailu'),
    limit: z.number().min(1).max(20).default(5),
  }),
  func: async ({ query, limit }) => { /* ... */ },
});
```

Typy pamięci w systemach agentowych

Pamięć odróżnia agentów użytecznych od agentów-jednorazówek. Bez niej agent nie może uczyć się z poprzednich interakcji ani korzystać z wiedzy zebranej w poprzednich sesjach.

Typ	Opis	Implementacja
Krótkoterminowa	Historia bieżącej rozmowy	Lista wiadomości w stanie agenta
Episodyczna	Zapisane sesje z przeszłości	MongoDB – kolekcja sesji
Semantyczna	Wiedza wyekstrahowana z doświadczeń	Baza wektorowa (MongoDB Atlas)

Implementacja pamięci konwersacyjnej w LangChain

```
const memory = new BufferMemory({
  chatHistory: new MongoDBChatMessageHistory({
    collection: db.collection('chat_history'),
    sessionId: `user-${userId}`,
  }),
  returnMessages: true,
  memoryKey: 'chat_history',
});

// Pamięć z automatycznym podsumowaniem przy przekroczeniu limitu tokenów
const summaryMemory = new ConversationSummaryBufferMemory({
  llm: new ChatOpenAI({ model: 'gpt-4o-mini' }),
  maxTokenLimit: 2000,
  returnMessages: true,
});
```

Przechowywanie historii w MongoDB zapewnia trwałość między restartami serwisu i umożliwia wznowienie rozmowy w nowej sesji.